

Signal to Noise Ratio for Uniform Quantizer

ECE 343

Project 1

Colin Gregg

February 4, 2025

Introduction

In this project I simulated the dependence of the SNR for quantization on the full-scale level of the Analog to Digital converter (X_m), and the root-mean-square value (RMS) of the sampled input. Quantization works by first defining an interval—later the partition—of where the value of the sampled input will be categorized.

Also, a new quantized interval—the codebook—maps the input values in a certain part of the partition to that value. This can be seen in the diagram below. We are also assuming the partition and codebook have uniform intervals

In this scheme, delta is defined by the number of bits used in the quantizer, which will provide the resolution of our output, and the full-scale level of the ADC, X_m . It is defined as:

$$\Delta = \frac{2X_m}{2^{B+1}} = \frac{X_m}{2^B}.$$

2^{B+1} arises from the fact that the book assumed the binary coding assigned to each level of the quantizer follows the 2's complement pattern:

Binary symbol	Numeric value, $\hat{x}_B$
0 _φ 1 1	3/4
0 _φ 1 0	1/2
0 _φ 0 1	1/4
0 _φ 0 0	0
1 _φ 1 1	-1/4
1 _φ 1 0	-1/2
1 _φ 0 1	-3/4
1 _φ 0 0	-1

With the 2's complement pattern, the most significant bit (MSB) is used to determine the sign, meaning our system's resolution is one less bit than expected. Therefore, we define a 2^{B+1} bit system.

Because we go from infinite resolution when sampling our input sequence, to a finite resolution in our quantizer, the two sequences will not always match. To calculate this error, we take the difference of the quantized sequence and our sampled input as:

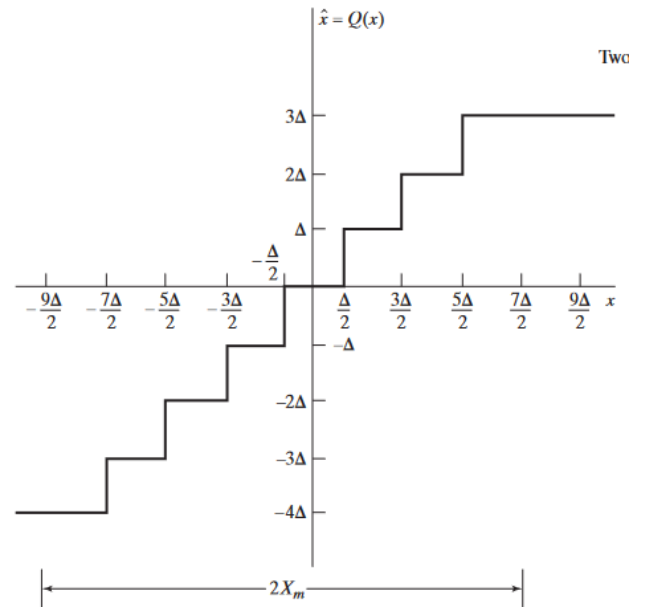


Figure 4.54 Typical quantizer for A/D conversion.

$$e[n] = \hat{x}[n] - x[n].$$

There is also another model of the error, where instead the system has the sampled input added with noise—i.e. the error—which produces the quantized output. The important characteristic of this model is that the error sequence has to be statistical in nature, and it follows a few core assumptions:

1. $e[n]$ is a sampled sequence of a stationary random process.
2. $e[n]$ is uncorrelated with the input sequence, $x[n]$.
3. The random variables that produce the error process are uncorrelated.
 - a. This means the error is white noise iff the joint distribution function of the random variables is gaussian.
4. The probability distribution of the error process is uniform over the range of the quantization error.

These, coupled with the fact the expectation of the error sequence, $E\{e[n]\}$, is equal to zero, produces the result that the standard deviation of the error sequence is simply the RMS of the sequence, and is equal to:

$$\sigma_e^2 = \int_{-\Delta/2}^{\Delta/2} e^2 \frac{1}{\Delta} de = \frac{\Delta^2}{12}.$$

Now that we have the variance of the error sequence, we can calculate the signal-to-noise ratio of the quantizer which is defined as:

$$\begin{aligned} \text{SNR}_Q &= 10 \log_{10} \left(\frac{\sigma_x^2}{\sigma_e^2} \right) = 10 \log_{10} \left(\frac{12 \cdot 2^{2B} \sigma_x^2}{X_m^2} \right) \\ &= 6.02B + 10.8 - 20 \log_{10} \left(\frac{X_m}{\sigma_x} \right). \end{aligned}$$

However, when calculating the SNR_Q in my project, I did not use this formula. When I attempted using this formula, it could not accurately model what happened at $(X_m/\sigma_x) < 1$, and the output did not show the increased resolution for higher bit values. Instead, I used this formula for SNR_Q :

$$SNR_Q = 10 \log_{10} \left(\frac{\frac{1}{N} \sum_{n=0}^{N-1} (x[n])^2}{\frac{1}{N} \sum_{n=0}^{N-1} (e[n])^2} \right),$$

Where SNR_Q is proportional to the log of the variances of each sequence. However, using properties of logs, and the fact that for our quantizer the variance is equivalent to the RMS value squared, I implemented the formula:

$$SNR = 20 \log_{10} \left(\frac{RMS(x[n])}{RMS(e[n])} \right)$$

Process

The general steps I followed to create my MATLAB script were:

- Define Constants
- Sample input cosine function
- Use quantiz() function to generate quantized sequence of input
- Calculate the error matrix
- Calculate the RMS of error and input sequence
- Calculate and plot SNR_Q

MATLAB Script

The constants I used for my program were:

```
%{***** Constants *****%}
B = [5 7 9 11 13 15]; % different bit values
N = 1000; % number of samples of cosine function
      % 100 samples per period of cosine
T = 200*pi; % sample for 10 periods of cosine
t = 0:(N/T):T; % equally spaced samples of cosine
Xm = 1;
%{*****%}
```

I found early on that the SNR was relatively invariant to the number of samples of the input, so I haphazardly chose 1000 as the number of samples. I sampled over 10 periods of cosine to ensure the samples were spaced enough. Little t is the array used to sample the cosine function and is 1000 equally spaced samples from 0 to 200π . X_m is a parameter of the physical quantizer—the ADC—which the book said to assume is normalized to 1.

Next, I defined the range of amplitudes I would be using. This is needed because the RMS of the input—the cosine function—is directly proportional to the amplitude of the cosine function. Therefore, to test how the SNR changes with the variances of the input signal, we need a vector of amplitude values. I choose the values to range from 0 to $2 \cdot X_m \cdot \sqrt{2}$, incrementing by 0.001. This allows us to get values of $1/A$ from about 0 to about 1000, which fits the figure 4.61.

After attaining the row vector for the amplitude values, I both sampled the input signal and applied an amplitude to it. This results in a matrix with dimensions of the length of the t vector by the length of the amplitude vector. To do this, I did element-wise multiplication of the A row vector and the $\cos(t/10)$ row vector. For my code, this came out to 395×2829 where every column represents one of the amplitudes and each row of that column is the sampled cosine value times the amplitude of that column.

The next step involves quantizing each of the 2829 input sequences, based on the number of bits used for quantization. This is done by utilizing the `quantiz()` function. This function works as:

```
[index, quantz] = quantiz(sig, partition, codebook);
```

Firstly, the partition defines the resolution with which the input signal will be split up. Looking at the image to the right, the partition is the x-axis. The codebook is the possible quantized values that the signal can be assigned, based on where in the partition that sample falls. This would be the y-axis of the picture to the right. Both the partition and the codebook are inherently defined by the number of bits in the quantizer in the form of delta. I defined delta, the partition, and the codebook as below:

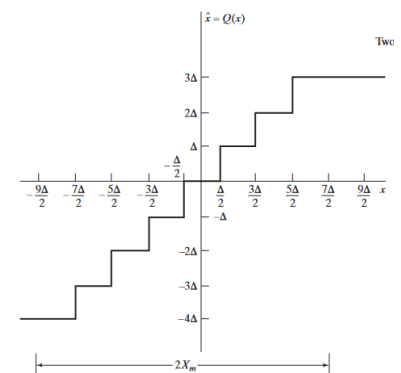


Figure 4.54 Typical quantizer for A/D conversion.

```
delta = Xm ./ (2.^(B));
```

```
% partition and codebooks for various number of bits
```

```
% 5 bits
```

```
partition_5 = -Xm : 2*delta(1) : Xm;
```

```
codebook_5 = -Xm - 2*delta(1) : 2*delta(1) : Xm;
```

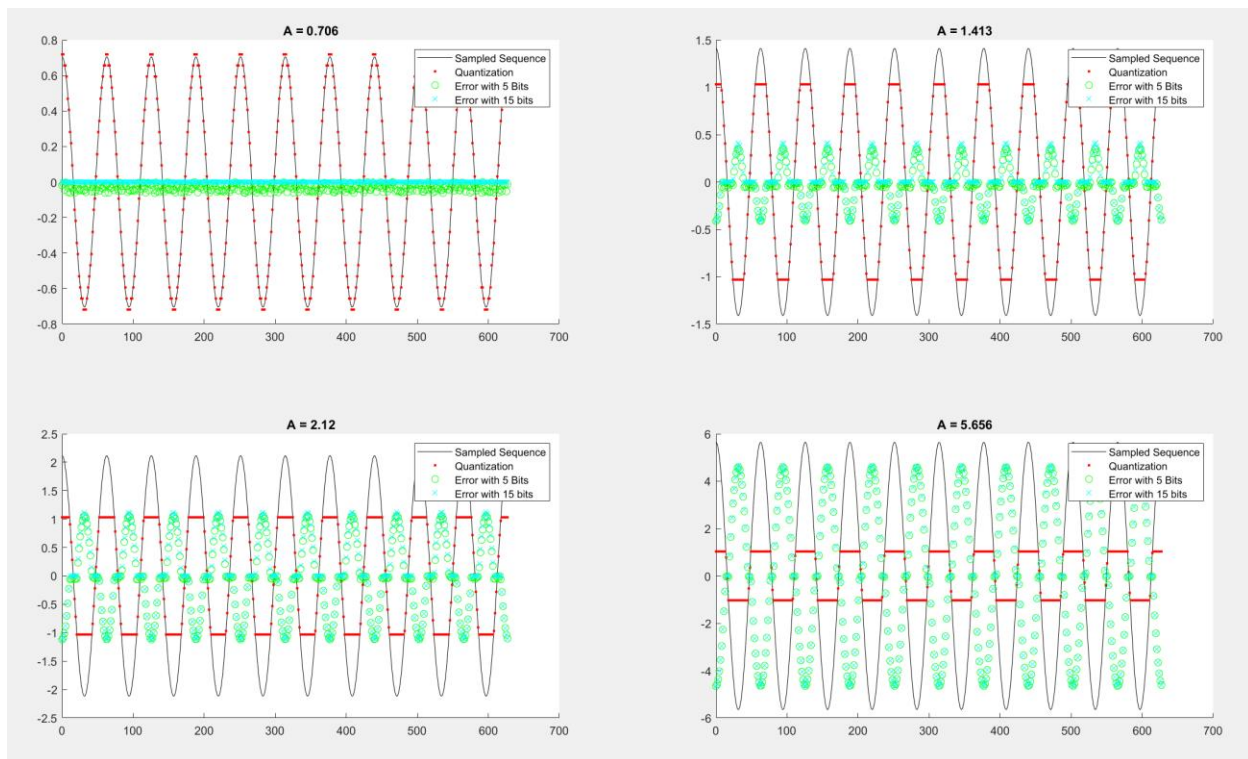
In regard to the output for quantiz(), I created a QUANTS matrix which contains the quants output for each bit value, and for each amplitude. Since the quants output vector will be the same length as the number of samples of the signal, the QUANTS matrix becomes a 395 x 2829 x 6 3D matrix. Because the index output is not necessary for this project I used a tilde to signify the output is not needed. The code for that is:

```
QUANTS = zeros(size(sig,1),length(A), length(B));
for i = 1:length(A)
    [~, QUANTS(:,i,1)] = quantiz(sig(:,i), partition_5, codebook_5); % 5 bits
    [~, QUANTS(:,i,2)] = quantiz(sig(:,i), partition_7, codebook_7); % 5 bits
    [~, QUANTS(:,i,3)] = quantiz(sig(:,i), partition_9, codebook_9); % 5 bits
    [~, QUANTS(:,i,4)] = quantiz(sig(:,i), partition_11, codebook_11); % 5 bits
    [~, QUANTS(:,i,5)] = quantiz(sig(:,i), partition_13, codebook_13); % 5 bits
    [~, QUANTS(:,i,6)] = quantiz(sig(:,i), partition_15, codebook_15); % 15 bits
end
```

I then calculated the error by subtracting the sampled input from the respective quantized sequence:

```
ERROR = zeros(size(sig,1), length(A), length(B));
for i = 1:length(B)
    for j = 1:length(A)
        ERROR(:, j, i) = QUANTS(:, j, i) - sig(:,j);
    end
end
```

Looking at the graphs for four of the sequences we can note a few things (graphs below). Firstly, the black line is the sample input, the red is the quantized values, the green circles are the error when 5 bits were used in the quantizer, and the cyan ex's are the error when 15 bits were used. We can see that as the amplitude increases, more of the quantization is cut off for the higher values of the input, and a corresponding spike in the error can be seen at those peaks or troughs. We can also see that the 15-bit error sequence is generally straighter than the 5-bit error sequence—i.e. less variance. Also note that while the amplitude is below $X_m \cdot \sqrt{2}$, the error is very minimal, while the opposite is true when the amplitude is greater than $X_m \cdot \sqrt{2}$.



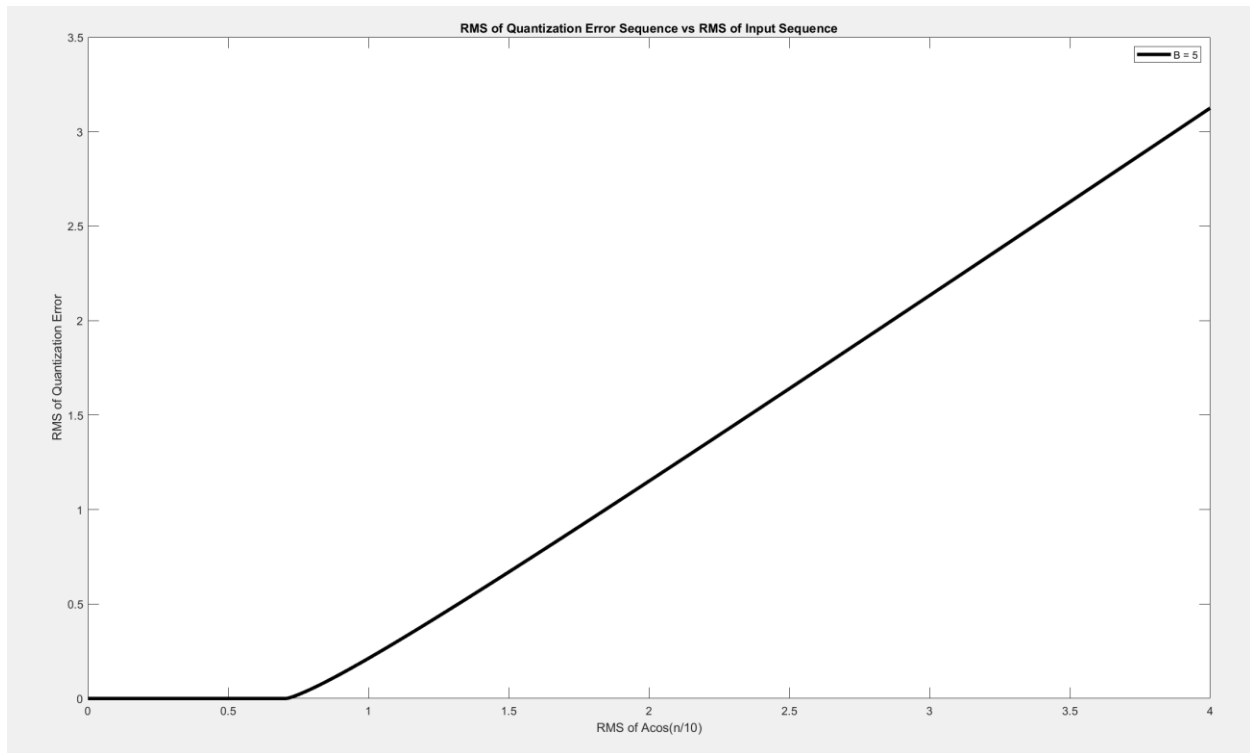
The next bit of code calculated the RMS values for each sequence in the sig and ERROR matrices. I also created a variable called log_ratio which was the ratio of RMS(sig) to RMS(error). It is the fraction used to calculate the SNR_Q . Because the input is a cosine function, the RMS of it is simply the amplitude divided by $\sqrt{2}$. I utilized the rms() function in matlab for finding the RMS of each sequence in the ERROR matrix.

```

RMS_error = zeros(length(A), length(B));
RMS_input = zeros(length(A), length(B));
log_ratio = zeros(length(A), length(B));
for i = 1:length(B)
    for j = 1:length(A)
        RMS_error(j,i) = rms(ERROR(:,j,i));
        RMS_input(j,i) = A(j) / sqrt(2);
        log_ratio(j,i) = ( A(j) ) / ( sqrt(2) * rms(ERROR(:,j,i)) );
    end
end
end

```

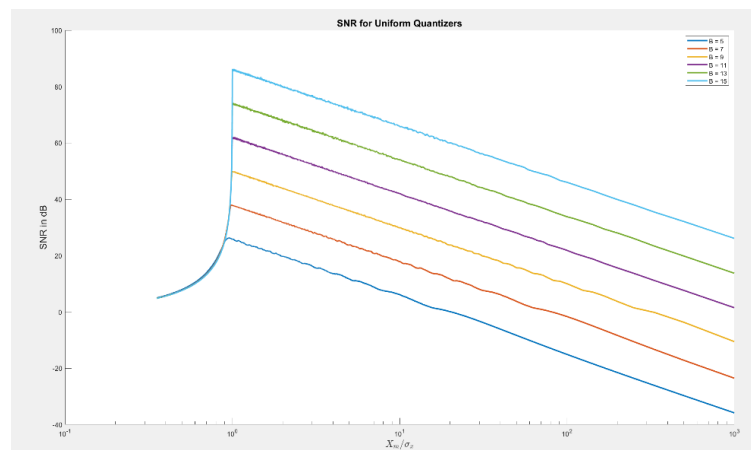
As seen below, a graph of the RMS of the error as a function of the RMS of the input—which is directly correlated to the amplitude—verifies what we saw above, that the variance of the error increases as the amplitude increases.



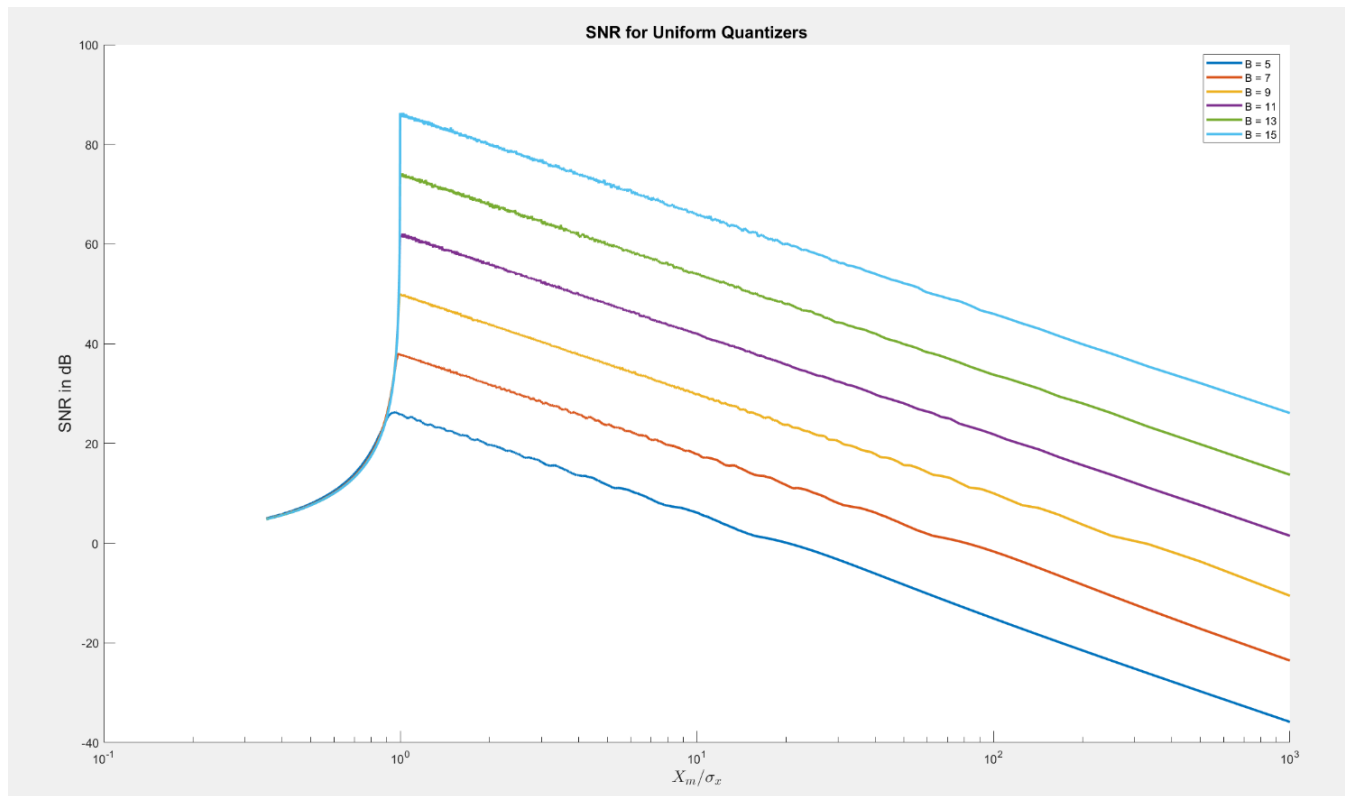
Lastly, I calculated and graphed the SNR_Q as a function of X_m/σ_x using this script:

```
figure(3)
hold on
for i = 1:length(B)
    plot((1./A), 20*log10(log_ratio(:,i)), '-', 'LineWidth',2);
end
hold off
```

Which produced the following graph:



Output



In this figure, we can see that for each of the ascending graphs, the number of bits increases by two. Also, the x-axis is in the logarithmic scale. Taking a closer look at the graph of the SNR for a uniform quantizer, there are a few things we can discern.

As the number of bits increases:

- The SNR increases as well
- The graph for SNR becomes less “wobbly” and forms a straighter line

As the ratio, X_m to σ_x becomes equivalent to 1, the SNR for each bit level shoots up to a high. This is very representative of both the distortion caused by too large of an input signal into the quantizer for $X_m/\sigma_x < 1$, and the equation for SNR of:

$$= 6.02B + 10.8 - 20 \log_{10} \left(\frac{X_m}{\sigma_x} \right)$$

which shows the SNR reaches a maximum when $\log_{10}(X_m/\sigma_x)$ equals zero, or $X_m/\sigma_x = 1$. We can also see that once $X_m/\sigma_x > 1$, the SNR decreases logarithmically as expressed by the $-20 \cdot \log_{10}(X_m/\sigma_x)$ value.